CTF Challenge Write-Up

Problem ID	Captured Flag	Steps (Summary)
P1 – one_byte	`fsuCTF{0h_n0_r3v3r5i813_3ncryp7i0n}`	Base64 decode ciphertext, brute-force all 256 single-byte XOR keys, check output for flag format
P2 – My Public Key	`fsuCTF{A_Big_N_Does_Not_Matter_If_Its_Factors_Are_Known}`	Factor small 512-bit RSA modulus into p and q, compute phi and private key d, decrypt ciphertext
P3 – The Cooler Cipher	`fsuCTF{You_Know_What_ROT13_Aint_Cool_Enough_Have_ROT47}`	Recognize hint pointing to ROT47, apply ROT47 rotation across all printable ASCII characters

P1 – one byte

The challenge provided a ciphertext that was Base64 encoded. After decoding, the data appeared as non-readable bytes. The challenge name 'one byte' was a direct hint that single-byte XOR encryption was used meaning every byte of the plaintext was XOR'd with the same key.

Analysis

Single byte XOR is trivially reversible by brute force since there are only 256 possible key values. XOR is also its own inverse, so the same operation decrypts as encrypts:

```
plaintext = ciphertext XOR key

# Since key is a single byte, try all 256 values
for key in range(256):
    result = bytes([b ^ key for b in decoded])
    if b'fsuCTF' in result:
        print(result)
```

Solution

1. Base64 decode the provided ciphertext
2. Brute force all 256 possible single-byte XOR keys
3. Check each result for the flag format fsuCTF{...}
4. The matching result is the flag

Flag: **fsuCTF{0h_n0_r3v3r5i813_3ncryp7i0n}**

P2 – My Public Key

This challenge used RSA encryption but with a weak 512 bit modulus. RSA security depends on the difficulty of factoring the modulus n into its two prime factors p and q. A 512 bit modulus is small enough to be factored with modern tools, breaking the encryption entirely.

Attack: Factoring the Modulus

Once p and q are known, the private key d can be computed and the ciphertext decrypted:

```
# Factor n (using a tool like factordb or msieve)
p, q = factor(n)

# Compute private key
phi = (p - 1) * (q - 1)
d   = pow(e, -1, phi)   # modular inverse of e

# Decrypt
m   = pow(c, d, n)
flag = m.to_bytes((m.bit_length() + 7) // 8, 'big').decode()
```

Solution

1. Extract n, e, and c from the provided public key and ciphertext
2. Factor n into p and q using factordb or a factorization tool (feasible at 512 bits)
3. Compute phi(n) = (p-1)(q-1)
4. Compute the private exponent d = e^-1 mod phi(n)
5. Decrypt the ciphertext: m = c^d mod n
6. Convert the resulting integer to bytes to get the flag string

Flag: **fsuCTF{A_Big_N_Does_Not_Matter_If_Its_Factors_Are_Known}**

P3 – The Cooler Cipher

The challenge name and hint referenced ROT13 but implied something stronger. ROT13 only rotates the 26 letters of the alphabet by 13 positions. The 'cooler' version is ROT47, which rotates across all 94 printable ASCII characters (codes 33 to 126).

Analysis

ROT47 works by shifting each printable ASCII character by 47 positions within the printable range, wrapping around at the end:

```
def rot47(text):
    result = []
    for c in text:
        if 33 <= ord(c) <= 126:
            result.append(chr(33 + (ord(c) - 33 + 47) % 94))
        else:
            result.append(c)
    return ''.join(result)
```

Solution

1. Recognize from the hint that ROT13 is being referenced but is not the correct cipher
2. Identify 'cooler cipher' as ROT47, which covers all printable ASCII (codes 33-126)
3. Apply ROT47 to the ciphertext to decode it
4. The decoded output is the flag

Flag: **`fsuCTF{You_Know_What_ROT13_Aint_Cool_Enough_Have_ROT47}`**